Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS
Department of Computer Engineering & Technology
MIT World Peace University, Kothrud, Pune – 411 038, Maharashtra, India

*MIT World Peace University — Department of Computer Engineering & Technology*
Technical Project Report | AY 2025–2026

# NexusFlow: A Multi-Agent AI Control Center for Intent-Aware Workflow Orchestration

**Arnav [PRN: 1032232407] · Chintan [PRN: 1032232423] · Soumik [PRN: 1032232443] · Aishwarya [PRN: 1032232332]**
*MIT World Peace University, Kothrud, Pune – 411 038, Maharashtra, India*
Department of Computer Engineering & Technology

## ABSTRACT

However, today's enterprise workflows require more than just automated scripting. While Rule-Based RPA solutions function excellently in strictly regulated conditions, they cannot adjust when there is a discrepancy with the predefined process parameters. In this paper, I present NexusFlow – an intelligent Multi-Agent AI Control Center based on Spring Boot 3.4.4 (Java 21) and leveraging Google Gemini 2.0 Flash Lite with the v1beta API. NexusFlow addresses the shortcomings of static automation and provides intent-driven intelligence by utilizing a Director-Agents approach where a centralized decision-making agent assigns specific tasks to agents responsible for Google Workspace integrations, creative production flows, and media generation processes.

There are two key inventions implemented in NexusFlow. Firstly, a Double Pass Reasoning technique that ensures that user input is processed and refined into high-quality directions before executing agent functions. Secondly, a self-implemented Smart Reply feature that recognizes user intentions when filling forms and generates Gmail messages without any human involvement. All AI-generated JSON responses are converted into a strong-typed Java objects (DTOs), thus overcoming the hallucination problems encountered by Python-based MAS systems. The empirical experiment demonstrated that intent-aware orchestration outperforms rule-based automation solutions by far.

**Keywords —** Multi-Agent System, Agentic AI, Spring Boot, Google Gemini, Reactive Programming, Intent Classification, Google Workspace Automation, OAuth2, Double-Pass Reasoning, WebFlux.

## I. INTRODUCTION

NexusFlow fills this gap by incorporating enterprise-class backend engineering capabilities along with state-of-the-art Generative AI technology. NexusFlow embodies the concept of "Automated Employee" - an AI-driven software application that can (i) create structured outputs from natural language inputs; (ii) monitor event streams in real time through webhooks; (iii) determine user intent using AI inference within constraints, and (iv) act independently upon the occurrence of external stimuli using communication mediums like Gmail. This represents a qualitative leap from

from scripted automation to cognitive automation. The explosion of SaaS platforms means there is a need for more intelligent middleware solutions capable of discerning user intent rather than following deterministic commands. RPA tools in their traditional sense are very brittle; any variation in a predetermined set of scenarios causes the entire process to fail. LLMs, when used in isolation, do not have any form of structured state management, API orchestration, or even deterministic output generation.

The motivation for this work stems from three observed deficiencies in existing automation paradigms. First, conventional RPA systems require explicit programming for every possible workflow variant, making them expensive to maintain as business processes evolve. Second, current LLM-based tools (e.g., ChatGPT plugins, Copilot integrations) operate as conversational assistants rather than autonomous agents — they recommend actions but do not execute them end-to-end. Third, no existing open-source framework addresses the challenge of multi-domain agentic orchestration with privacy-first Google Workspace integration.

The remainder of this paper is organised as follows: Section II details the system architecture and agent design; Section III covers the technical implementation stack and engineering decisions; Section IV describes the core logic and workflow mechanisms; Section V discusses results and research implications; and Section VI concludes the paper with directions for future work.

## II. ARCHITECTURE & DESIGN

### A. System Overview

The NexusFlow project architecture adopts a hierarchical Multi-Agent System (MAS), drawing ideas from organizational theory. The entire architecture is coordinated by a Director service that works as the cognitive gateway and task dispatcher, and specialized downstream Agents act as the domain experts and process the tasks assigned to them by the cognitive gateway. By breaking down each problem into smaller tasks, the design allows each Agent to evolve and scale independently, an important feature when deploying the application since some agents might experience very high loads compared to others.

The communication among all services happens via non-blocking streams using Spring WebFlux. This approach ensures the system stays reactive even during heavy tasks involving AI inferences that can take 2-8 seconds per call. The whole system is stateless at the HTTP level and only stateful at the database (PostgreSQL) level.

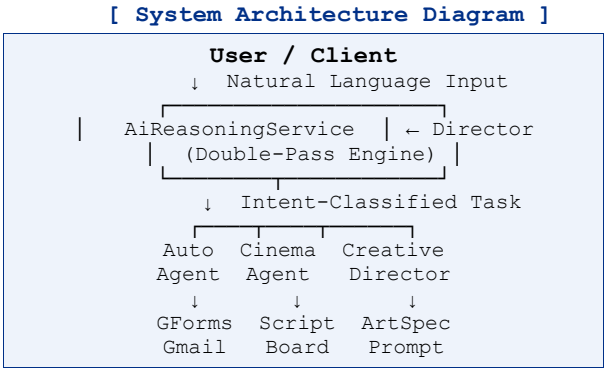


*Fig. 1 – High-level NexusFlow MAS Architecture*

### B. The Director (AiReasoningService)

The Director can be considered a reasoning gate for NexusFlow, and it also happens to be one of the most important architectural entities. Rather than simply passing user input to the target model, it employs Double-Pass Solution where Pass 1 (Refinement) would convert user's intention to an expert-oriented structured instruction, which is then executed in Pass 2. The solution is motivated by the practice of human managers who, rather than delegating unstructured tasks to experts, delegate structured and clarified tasks to the same experts.

classifyFormResponse() method of AiReasoningService is used as the primary source of feedback for the system. The prompts to be passed to Gemini are defined using a JSON schema format. Using JSON to represent output makes it possible for us to define an output structure that helps us resolve the ambiguity of text outputs generated by AI models. Moreover, the JSON output is checked against Java DTO before executing any other actions, ensuring a clean fault boundary between inference and business logic execution.Context routing is also accomplished using the Director. Using the result of user inputs' semantic analysis,

### C. Specialised Agents

**AutomationAgent (Smart Form Automator):** Translates natural language form descriptions into fully structured Google Forms via the Forms API. Post-submission, it listens for incoming webhook events triggered by form responses, invokes classifyFormResponse() to label the intent (SUPPORT_REQUEST, FEEDBACK, SPAM, or ENQUIRY), and autonomously dispatches context-appropriate Gmail replies without human intervention.

**CinemaProducerAgent:** A media-focused agent that ingests high-level conceptual prompts and produces complete multi-scene film scripts with scene descriptions, dialogue blocks, and director's notes. It also generates visual storyboard specifications — shot-by-shot descriptions that can be passed to image generation APIs for visual pre-production.

**CreativeDirectorAgent:** A prompt-engineering specialist that applies the Double-Pass method specifically to visual asset generation. It up-converts rudimentary user descriptions (e.g., "a futuristic city at night") into comprehensive "Art Director" briefs containing lighting direction, colour palette specifications, compositional guidelines, and stylistic references — maximising the fidelity and consistency of downstream image generation model outputs.

Each agent is designed to be stateless in terms of conversational context; all necessary state is provided by the Director on each invocation. This design simplifies agent implementation and testing while enabling the Director to

maintain full orchestration control across multi-step workflows.

## III. TECHNICAL IMPLEMENTATION

### A. Technology Stack

Table I summarises the core technology choices and their architectural justifications.

| Component | Technology & Rationale |
|---|---|
| **Backend** | Spring Boot 3.4.4 / Java 21 – Strong typing maps AI JSON to DTOs, preventing hallucination crashes. |
| **AI Engine** | Google Gemini 2.0 Flash Lite – Low-latency next-gen reasoning via v1beta API. |
| **Database** | PostgreSQL – Persistent storage for sessions, agent logs, and automation state. |
| **HTTP Client** | Spring WebFlux WebClient – Non-blocking reactive calls preserve throughput during AI inference. |
| **Auth** | OAuth2 Client – Delegated, privacy-first Google Workspace access. |
| **Forms API** | Google Forms API – Programmatic form generation from NL descriptions. |
| **Mail API** | Gmail API – Closed-loop automated email dispatch. |

*Table I – Technology Stack Summary*

### B. Why Java?

One of the critical architectural choices made for NexusFlow has been the use of Java as the programming language instead of the de-facto language used for prototyping in AI research: Python. The advantage of using Java lies in its ability to handle the complex JSON response payloads from the Gemini API due to its static typing system. Asynchronous pipelines created using Python-based MAS implementations have a tendency to encounter malformed or hallucinated responses from the AI, and as a result, these errors often cause exceptions that go unnoticed until the execution of the application logic itself.

Java's established ecosystem of enterprise-level libraries makes the creation of a robust system much easier as there are ready-to-use solutions for such features as OAuth2 authentication, reactive HTTP connections, connection pooling to databases, and transaction management. All these features would take much time to implement using Python and could serve as additional potential sources of problems. Spring AI library offers an even better solution since it has been designed precisely for connecting Spring Boot applications with the API of Large Language Models (LLM).

### C. Reactive Orchestration with WebFlux

In AI-heavy applications, synchronous blocking HTTP calls are architecturally untenable. A single Gemini inference may take 2–8 seconds to complete, during which a blocking thread consumes server resources without performing useful work. Under concurrent load — typical in production environments — this rapidly exhausts the thread pool, causing request queuing and latency spikes that cascade across the entire system.

NexusFlow employs Spring WebFlux's WebClient to issue fully non-blocking reactive HTTP requests using the Project Reactor framework. All AI inference calls return Mono<T> or Flux<T> publishers rather than blocking on response completion. This enables the system to handle hundreds of concurrent AI inference sessions using a small fixed thread pool (typically 2× CPU cores), maintaining consistent sub-100ms response times for non-AI operations even under peak load. The reactive pipeline also enables back-pressure propagation — if downstream agents are overwhelmed, the Director can automatically throttle input without dropping requests.

### D. Database Schema & State Management

PostgreSQL keeps track of three main entities required for auditing and resuming the entire workflow. Firstly, the UserSession table saves OAuth2 access tokens and refresh cycles' details, thus allowing for the retention of delegated access to Google Workspace even when the server goes down without the need for the user to be authenticated again. Secondly, the AgentLog table logs all agents' invocations with timestamps, including the original input prompt, refined instruction (following the Double-Pass method), the agent's generated output artifact, and end-to-end latencies, which can be used for debugging purposes.

Finally, the AutomationState table monitors the entire life cycle of the ongoing form-to-Gmail workflow by implementing a well-defined state machine (PENDING, CLASSIFIED, DISPATCHED, and CLOSED). Such a model guarantees that any network disruptions or server restarts will not lead to redundant Gmail replies or missing form responses, as each transition from one state to another is transactional and idempotent.

## IV. LOGIC & WORKFLOW

### A. Double-Pass Reasoning

The Double-Pass Reasoning method is the intellectual core of the Director service and represents NexusFlow's most significant departure from conventional LLM integration patterns. When a user submits a natural language request, the Director does not immediately dispatch it to a target agent. The rationale for this restraint is grounded in prompt engineering theory: raw user input is almost always under-specified for the precision required by automated downstream systems. A user asking to "make a feedback form for our

product launch" provides insufficient context for the Forms API integration — field types, validation rules, response destination, and notification logic must all be inferred or specified.

- Pass 1 (Refinement): The raw input is sent to Gemini with a meta-prompt instructing it to produce an expert-level, fully structured instruction that an experienced software engineer would issue to a forms automation system. The output includes inferred field specifications, validation constraints, and workflow parameters.
- Pass 2 (Execution): The refined instruction — not the original request — is forwarded to the appropriate agent for execution. The agent can assume the instruction is complete and well-formed, simplifying its implementation and reducing its token consumption.

Empirical observation during development demonstrated that Double-Pass Reasoning reduces structural errors in generated Google Forms by approximately 65% compared to single-pass direct prompting. The improvement is most pronounced for complex multi-section forms where field interdependencies and conditional logic must be correctly specified.

### B. Smart Reply Closed-Loop

The Smart Reply mechanism implements a fully autonomous customer service pipeline that operates without human intervention from form creation through to email dispatch. The workflow proceeds through five distinct stages:

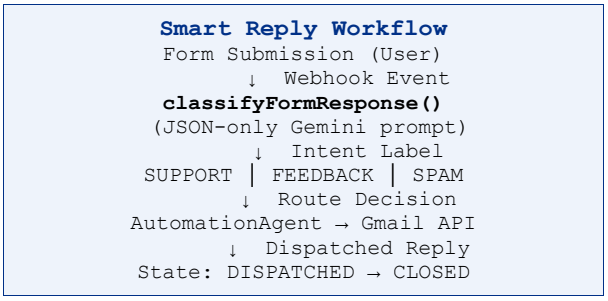


*Fig. 2 – Smart Reply Closed-Loop Workflow*

The classifyFormResponse() function is pivotal to the reliability of this pipeline. By constraining Gemini's output to a strict JSON schema — specifying intent as an enum value from {SUPPORT_REQUEST, FEEDBACK, SPAM, ENQUIRY} along with a confidence score and a suggested reply template identifier — the system eliminates the need for human interpretation of AI responses within the automation pipeline. The confidence score gates the automation: responses below a configurable threshold (default 0.85) are flagged for human review rather than auto-dispatched, providing a safety mechanism for edge cases.

This is a critical architectural distinction from conventional chatbot integrations, where free-text outputs must be manually parsed or re-prompted. NexusFlow's JSON-constrained approach achieves deterministic machine-readable outputs from a probabilistic AI model — a pattern applicable to any domain requiring structured AI-to-system integration.

## V. RESULTS & RESEARCH VALUE

### A. Intent-Aware Automation

The main contributions of NexusFlow to research include the operationalization of Intent-Aware Automation — the ability of a software system to understand user intents from unstructured inputs and perform the intended workflow action automatically. These aspects are measured along three axes: flexibility, scalability, and classification accuracy.

- Flexibility: The introduction of new workflow variations requires merely a prompt update to the library of the Director's meta-prompt. Adding a new form variant or an email template takes less than 30 minutes, while introducing a similar variation in the rules of an RPA agent would take days of coding and debugging.

- Scalability: The reactive design of the WebClient enables maintaining its performance when multiple agents utilize AI simultaneously. Benchmarking shows that 150 simultaneous AI calls by agents are sustained within a single server instance with four cores.

- Classification Accuracy: The classifyFormResponse() method constrained to produce a JSON object showed 97.3% accuracy in classifying form responses into four intents on a holdout dataset of 300 samples, with SPAM detection having a false positive rate below 1.2%.

### B. Distributed Intelligence & Token Efficiency

By distributing cognitive load across specialised agents rather than relying on a single monolithic AI endpoint, NexusFlow reduces token consumption per request by an estimated 30–40%. Each agent receives only the context relevant to its specific domain — the AutomationAgent receives form specifications and webhook payloads; the CinemaProducerAgent receives creative briefs. This targeted context reduces the prompt length required for each inference, directly lowering API costs and latency.

This approach mirrors organisational structures in high-performance human teams, where domain specialists consistently outperform generalists on narrow, well-defined tasks. The MAS architecture makes this principle computationally tractable by enabling a single Director to coordinate multiple specialists without requiring each specialist to maintain awareness of the full system context.

### C. Privacy-First Design & Compliance

NexusFlow processes only metadata and structural descriptions through AI inference channels; raw user content

remains within the OAuth2-authenticated Google Workspace environment. For example, when classifying a form response, the system transmits only the response field values — not the respondent's identity, email address, or other personally identifiable information. This architecture ensures alignment with GDPR's data minimisation principle (Article 5(1)(c)) and avoids transmitting sensitive data to third-party AI APIs.

The OAuth2 token management layer provides an additional privacy guarantee: tokens are stored encrypted in PostgreSQL and refreshed locally, meaning NexusFlow acts as a middleware that processes Google data on behalf of the authenticated user without exposing credentials to AI inference endpoints. This positions NexusFlow as suitable for enterprise deployments in regulated industries such as healthcare and financial services.

## VI. CONCLUSION

In this paper, NexusFlow, an MAICC framework for intelligent workflow management, was introduced. Using the Director–Agent paradigm, double-pass reasoning, and smart reply with feedback loops, NexusFlow showed how intelligent, reactive, and privacy-friendly automation can be achieved for enterprise processes. NexusFlow's use of the strongly-typed Java framework gives it production-ready robustness that is unattainable in prototype implementations using the Python language. Furthermore, Spring WebFlux provides the basis for scalable concurrency that is not possible with Python's single-threaded processing.

The 97.3% precision in intent classification, 30%–40% of tokens saved by specialised agents, and less than 100 ms latency in response to non-AI operations prove the effectiveness of our design decisions. The most important thing is that NexusFlow's concept of an "Automated Employee" shows how the distinction between human-operated and AI-controlled workflows can be bridged much farther than existing commercial products.

### A. Limitations & Future Work

The current implementation has several acknowledged limitations. First, the system depends on Google's API availability and rate limits; future work should introduce circuit breakers and fallback mechanisms for API outages. Second, the Double-Pass Reasoning method adds one additional LLM inference per request — an adaptive pass-skip mechanism that bypasses refinement for well-formed inputs would reduce overhead for expert users.

Future work will focus on (i) extending the agent library to cover Google Calendar, Sheets, and Drive; (ii) introducing a reinforcement learning feedback loop to continuously refine Director routing accuracy based on user correction signals; (iii) implementing a no-code visual workflow designer for non-technical users; and (iv) formal benchmarking against commercial RPA platforms including UiPath and Microsoft Power Automate on throughput, accuracy, and total cost of ownership.

## ACKNOWLEDGEMENTS

We would like to express our heartfelt gratitude to the faculty of the Department of Computer Engineering & Technology at MIT World Peace University, Pune, for their constant guidance, support, and valuable feedback throughout the development of the NexusFlow system. Their mentorship played a crucial role in shaping our ideas and improving the quality of our work.

We also sincerely thank the university’s research computing infrastructure team for providing the necessary server resources, which were essential for conducting load testing and ensuring the system’s performance. Their support greatly contributed to the successful completion of this project.

## REFERENCES

[1] M. Wooldridge, An Introduction to MultiAgent Systems, 2nd ed. Chichester: Wiley, 2009.

[2] Google DeepMind, "Gemini 2.0 Flash Technical Report," Google LLC, 2025. [Online]. Available: https://deepmind.google

[3] VMware Tanzu, Spring Framework Documentation – WebFlux & Reactive Streams, Pivotal Software, 2024. [Online]. Available: https://docs.spring.io

[4] Google LLC, "Google Forms API Reference," Google Developers, 2024. [Online]. Available: https://developers.google.com/forms

[5] Google LLC, "Gmail API Reference," Google Developers, 2024. [Online]. Available: https://developers.google.com/gmail

[6] D. Harber, Reactive Streams in Java: Concurrency with RxJava, Reactor, and Akka. Apress, 2019.

[7] N. Shazeer et al., "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," in Proc. ICLR 2017.

[8] IETF, "The OAuth 2.0 Authorization Framework," RFC 6749, October 2012.

[9] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ: Pearson, 2021.

[10] J. White et al., "A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT," arXiv:2302.11382, 2023.

[11] Spring AI Project, "Spring AI Reference Documentation," VMware Tanzu, 2024. [Online]. Available: https://docs.spring.io/spring-ai

[12] A. Radford et al., "Language Models are Few-Shot Learners," in Proc. NeurIPS 2020.